Redux Recap — Understanding the Core Flow

Before learning async logic in Redux, we must fully understand how Redux works in its synchronous form.
Redux is essentially a **predictable state management flow** built around a simple but powerful pipeline.

Redux follows one fundamental rule:

> **State is never changed directly. Instead, state is updated through actions processed by reducers.**

To understand this flow, let's break down the key parts using the diagram.

The Core Redux Pieces

Redux consists of four main elements:

1. Component
2. Action
3. Reducer
4. Store (State)

Together, they form a unidirectional data flow that ensures predictable updates and easier debugging.

The Redux Flow Overview

The Redux lifecycle can be summarized as:

```
None
User Event → Action → Reducer → State Update → Component Re-render
```

This is a continuous loop where UI interactions lead to state updates, which in turn update the UI.

1 Step One — The Event Happens in the Component

Redux always begins with a **user interaction** inside a React component.

Examples of events:

- Clicking a button
- Submitting a form
- Typing in an input
- Selecting a filter

When the event occurs, the component dispatches an action.

Example:

```javascript
<button onClick={() => dispatch(addItem(item))}>
```

Here, the component is requesting a state update but does not change the state itself.

This is a critical architectural decision in Redux:

> Components request state changes — they never perform them directly.

2Step Two — The Action is Created

An action is a plain JavaScript object that describes what happened.

It typically contains:

```javascript
{
  type: "cart/addItem",
  payload: item
}
```

Understanding Action Structure

Property	Meaning
type	Describes the event that occurred
payload	Carries the data needed for the update

Actions do not contain logic.
They simply **describe an event** in the system.

Think of an action as a message saying:

> "Something happened — please update the state accordingly."

3 Step Three — The Reducer Processes the Action

The reducer is the heart of Redux.

It receives:

- the current state
- the action

And returns a **new state**.

Conceptually:

```
None
currentState + action → reducer → newState
```

Example (classic Redux):

```javascript
function reducer(state, action) {
  switch (action.type) {
    case "cart/addItem":
      return {
        ...state,
        items: [...state.items, action.payload]
      };
    default:
      return state;
  }
}
```

Important Reducer Rules

Reducers must be:

- Pure functions
- Synchronous
- Deterministic
- Free from side effects

That means reducers cannot:

- Call APIs
- Use timers
- Access local storage directly
- Generate random values
- Perform async operations

This constraint ensures predictable state updates.

4 Step Four — The Store Updates the State

After the reducer produces the new state, Redux replaces the old state with the new one.

In Redux:

> State is never mutated — it is replaced.

Redux Toolkit uses Immer, allowing code that looks like mutation:

```javascript
state.items.push(item);
```

But internally it remains immutable.

5 Step Five — Components Re-render

React Redux connects components to the store through subscriptions.

This is done using:

```javascript
useSelector()
```

When the state changes:

- subscribed components are notified
- React re-renders them
- UI reflects the new state

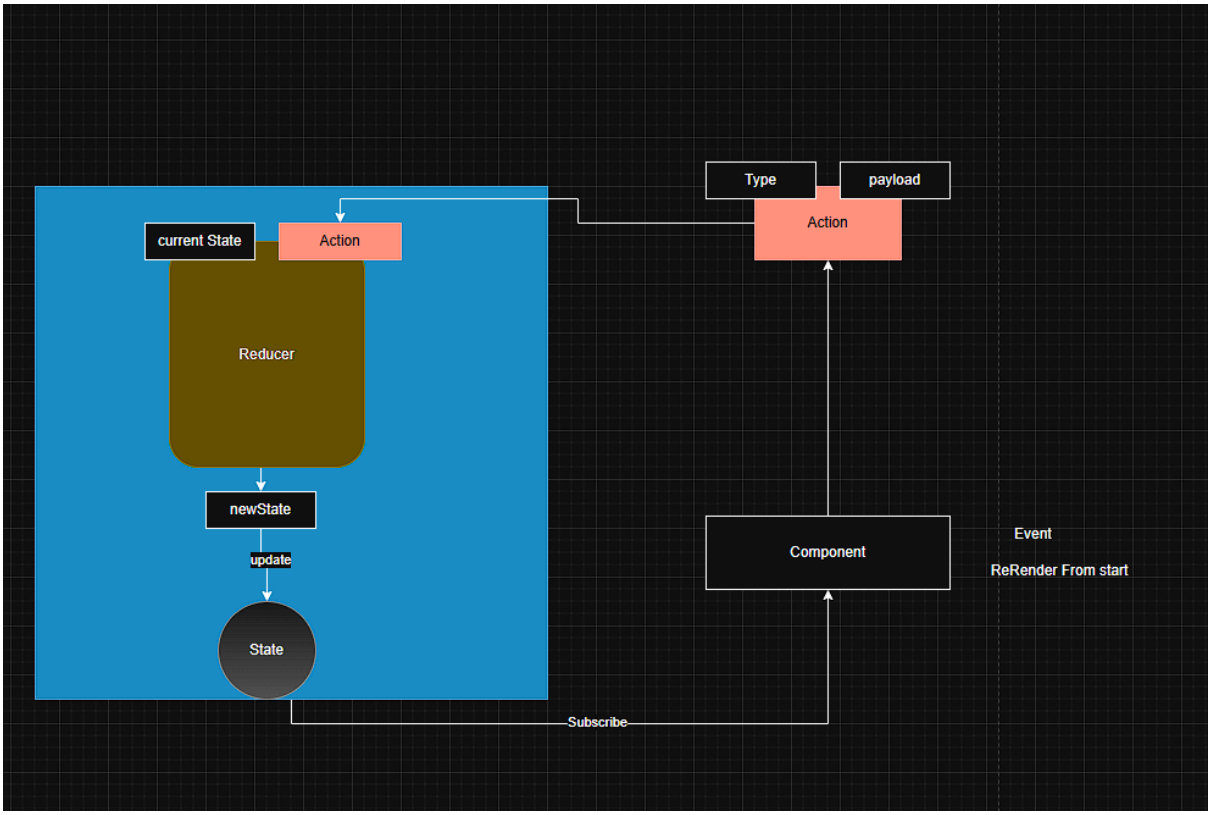
This automatic update is what makes Redux powerful and predictable.

The Redux Loop in Words

We can describe the Redux cycle as:

1. The user performs an action in the UI.
2. The component dispatches an action.
3. Redux sends the action to the reducer.
4. The reducer calculates a new state.
5. The store updates the state.
6. React components subscribed to the state re-render.

And the loop continues.



Async Operations in Redux

After understanding the normal Redux flow, we now move to one of the most important topics in modern applications:

Asynchronous operations

These are operations that do not complete immediately and require waiting for external processes.

1 What is an Async Operation?

An asynchronous operation is any task that:

- Takes time to complete
- Happens outside the immediate execution flow
- Returns its result later

Common examples in frontend applications:

- API requests
- Database calls
- Authentication requests
- File uploads
- Timers
- WebSocket events

Example:

```javascript
const res = await fetch("/api/products");
```

The UI must wait for the server to respond.

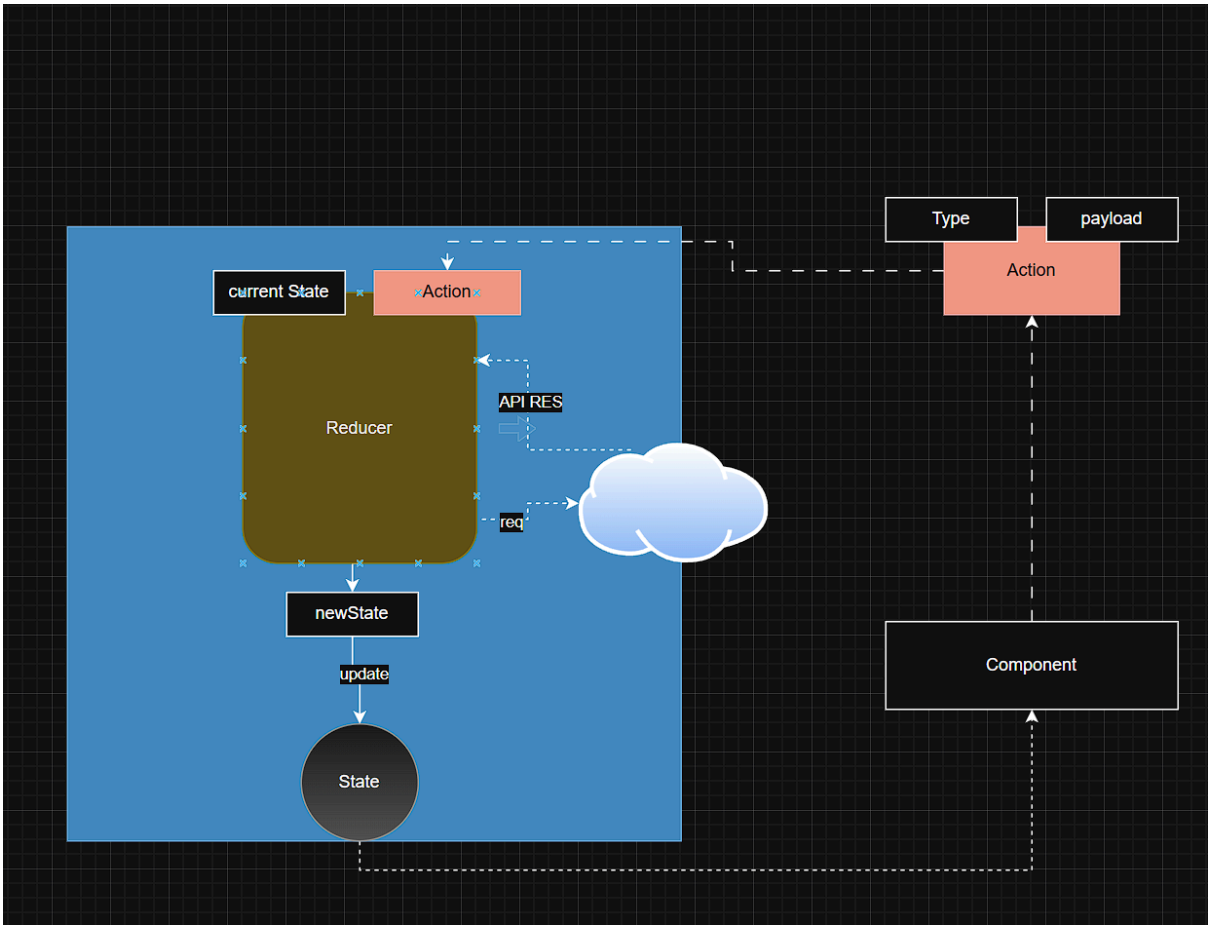
2 Why Async Exists in Modern Apps

Modern applications are data-driven.
They constantly communicate with external systems:

- Backend servers
- Cloud databases
- Authentication providers
- Third-party APIs

Because these systems take time to respond, async behavior is unavoidable.

So async is not a special case — it is the **default reality of real apps**.



3 The Problem: Redux Reducers Cannot Handle Async

Your diagram illustrates a common mistake: trying to perform API calls inside reducers.

Reducers must be:

- Pure
- Synchronous
- Deterministic
- Side-effect free

This means reducers cannot:

- Fetch data
- Wait for promises
- Use timers
- Access external systems

Why?

Because Redux depends on predictability.

If reducers were async:

- State updates would become unpredictable
- Debugging would break
- Time-travel debugging would fail
- State history would be unreliable

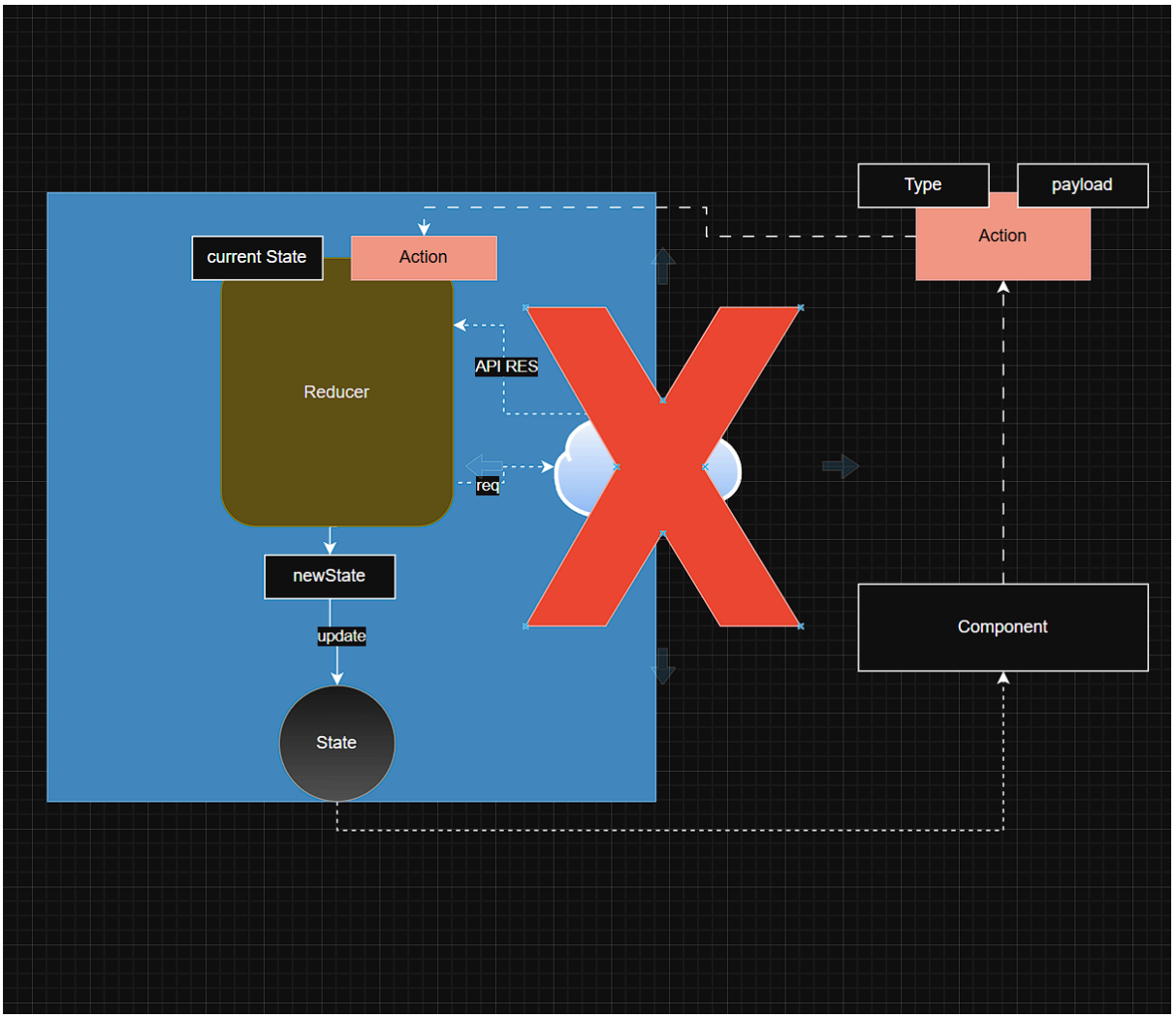
So reducers are intentionally restricted.

4 What Happens If We Try Async in a Reducer?

Trying to call an API inside a reducer creates:

- Hidden side effects
- Non-deterministic behavior
- Race conditions
- Broken Redux guarantees

Reducers must remain pure state calculators.



The Async Redux Approach — Middleware & Thunk Architecture

After understanding that reducers must remain synchronous and pure, we now explore how Redux handles asynchronous operations without breaking its core principles.

The solution is based on introducing an additional layer in the Redux pipeline:

Middleware

The Core Idea

Redux does not execute async logic inside reducers.
Instead, async logic happens **before reducers** using middleware.

This preserves Redux's predictability while allowing real-world async workflows.

The Extended Redux Flow (Async Version)

```
None
Component → Thunk Function → Middleware → API Request →
Dispatch Action → Reducer → State Update → UI Re-render
```

TStep One — Component Dispatches a Thunk

In a normal Redux flow, the component dispatches an action object.

But in async Redux, the component dispatches a **function**:

```javascript
dispatch(fetchProducts())
```

Here, `fetchProducts` is a thunk function.

This is different from traditional Redux because Redux normally expects plain objects.

Step Two — Middleware Intercepts the Thunk

Redux middleware sits between dispatch and reducers.

When middleware sees a dispatched function instead of an action object, it recognizes it as a thunk.

Instead of passing it to the reducer, middleware executes the function.

This is why middleware is essential for async Redux.

Step Three — Thunk Executes Async Logic

The thunk function performs the async operation:

- API request
- Database call
- Timer
- Authentication
- Any side effect

Example:

```javascript
async (dispatch) => {
  const res = await fetch("/api/products");
  dispatch({ type: "products/loaded", payload: data });
}
```

At this stage, Redux reducers are not involved yet.

The thunk prepares the data.

Step Four — API Request Happens Outside Redux Core

Your diagram shows the API cloud outside the reducer.

This is intentional.

Redux architecture separates:

- state calculation (reducers)
- side effects (thunks)

This separation ensures predictable state transitions.

Step Five — Thunk Dispatches Lifecycle Actions

After the async operation completes, the thunk dispatches actions back into Redux.

These actions represent the lifecycle:

1Pending → request started
2 Fulfilled → success
3 Rejected → failure

Redux Toolkit automates this lifecycle using `createAsyncThunk`.

Step Six — Reducer Receives Normal Actions

Once the thunk dispatches actions, Redux returns to its normal synchronous flow.

Reducers process these actions and update state.

The reducer does not know or care about async — it only reacts to actions.

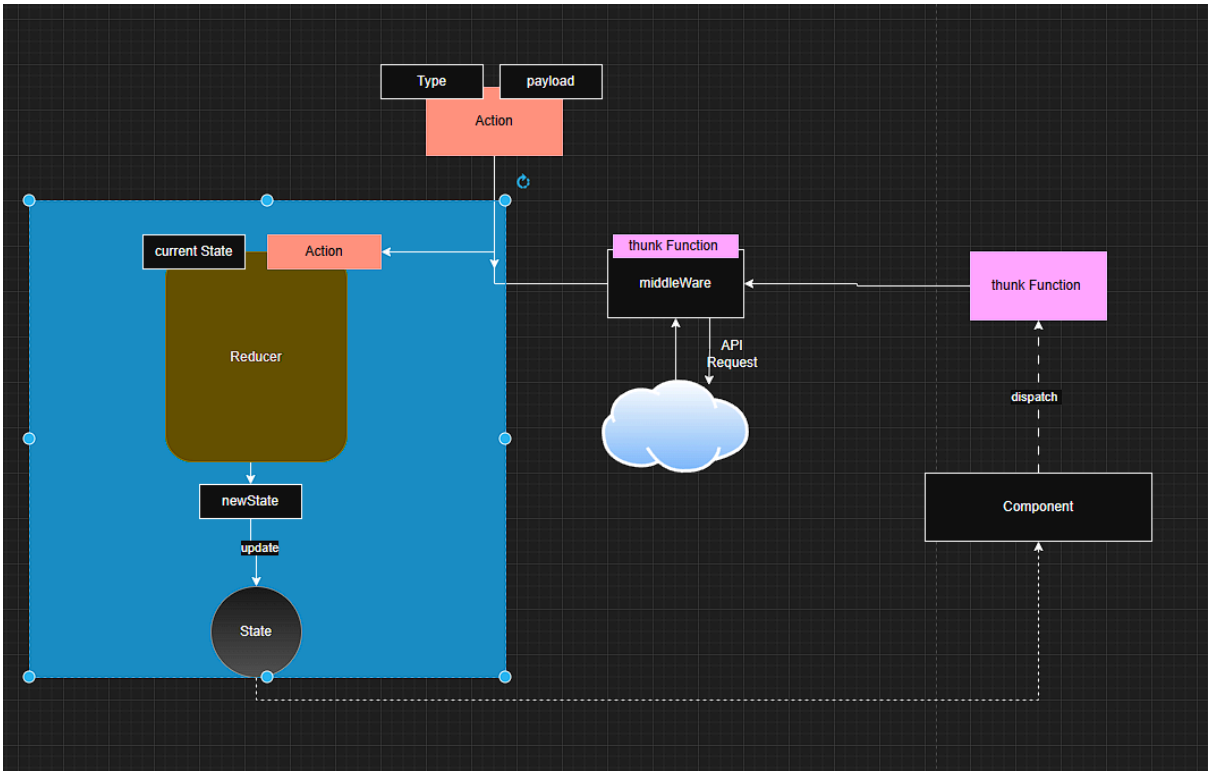
This keeps reducers pure.

Step Seven — Store Updates and UI Re-renders

After the reducer updates the state:

- Store replaces the old state
- React Redux notifies subscribed components
- UI re-renders automatically

This completes the async Redux loop.



Async Redux Documentation — Using createAsyncThunk

Step 1 — Create a folder for async logic

Inside your feature folder, create a dedicated file to hold async logic.

Example structure:

```
None
features/
  products/
    productsThunk.js
    productsSlice.js
```

Why do we separate thunks from slices?

Because async logic and state logic are different responsibilities.

Step 2 — Import createAsyncThunk

Inside `productsThunk.js`, we start with:

```
JavaScript
import { createAsyncThunk } from "@reduxjs/toolkit";
```

What is createAsyncThunk?

`createAsyncThunk` is a helper provided by Redux Toolkit that simplifies async logic.

It allows us to:

- Execute async operations
- Automatically generate lifecycle actions
- Handle loading and error states
- Integrate async logic with slices easily

Why do we need createAsyncThunk?

Remember: reducers must be pure and synchronous.

That means reducers cannot:

- call APIs
- wait for promises
- perform side effects

So we need a place to run async code **before reducers update state**.

That place is the thunk.

And `createAsyncThunk` is the official Redux Toolkit way to create thunks.

Step 3 — Create the thunk

```javascript
export const fetchData = createAsyncThunk(
  "feature/fetchData",
  async (_, thunkAPI) => {
    // async logic here
  }
);
```

Understanding the createAsyncThunk syntax

1 The type prefix

```javascript
"feature/fetchData"
```

This is just a naming convention.

Redux Toolkit uses it to generate lifecycle actions automatically:

```none
feature/fetchData/pending
```

```
feature/fetchData/fulfilled
feature/fetchData/rejected
```

These actions represent the async lifecycle.

2 The async function (payload creator)

```javascript
async (_, thunkAPI) => {}
```

This function contains the async logic.

It receives two parameters:

arg

The value passed when dispatching the thunk.

Example:

```javascript
dispatch(fetchData(10))
```

Here:

```
arg = 10
```

If unused → use `_`.

thunkAPI

An object containing helper functions:

- dispatch → dispatch other actions
- getState → read Redux state
- rejectWithValue → custom error payload
- signal → cancel request
- extra → dependency injection

Step 4 — Write async logic inside the thunk

```javascript
export const fetchData = createAsyncThunk(
  "feature/fetchData",
  async (_, thunkAPI) => {
    try {
      const res = await fetch("YOUR_API_ENDPOINT");

      if (!res.ok) {
        return thunkAPI.rejectWithValue("Server error");
      }

      const data = await res.json();
      return data;
    } catch (error) {
      return thunkAPI.rejectWithValue(error.message);
    }
  }
);
```

What happens when we return data?

When we write:

```javascript
return data;
```

Redux Toolkit automatically dispatches:

```
feature/fetchData/fulfilled
```

And:

```
action.payload = data
```

What happens when we reject?

When we write:

```javascript
return thunkAPI.rejectWithValue("Error message");
```

Redux Toolkit dispatches:

```
feature/fetchData/rejected
```

And:

```
action.payload = "Error message"
```

Why not throw an error?

If we throw:

```
None
throw new Error("x")
```

Redux puts the error in:

```
None
action.error.message
```

But `rejectWithValue` gives us:

```
None
action.payload
```

Which is cleaner for UI usage.

The internal lifecycle of a thunk

When dispatching:

```
JavaScript
dispatch(fetchData())
```

Redux Toolkit does:

1 dispatch pending
2 run async function
3 if success → dispatch fulfilled
4 if failure → dispatch rejected

Reducers only react to these actions.

Key takeaway

`createAsyncThunk` does not update state.

It only:

- runs async logic
- dispatches lifecycle actions

State updates happen inside the slice.